```
23.  Name stdout
24.  Format json_lines
25.  Match *
```

Now the configuration file is ready and we can start Fluent Bit to see our pipeline in action. To run it, assuming we are inside the build directory, execute the following command:

```
$ bin/fluent-bit -c ../conf/my_conf.conf
```

Figure 5 shows the output of the data pipeline setup.

As you can see, our records are being printed to the standard output after getting modified, i.e., after we add a tag. So far, we discussed that Fluent Bit offers multiple plugins and features that allow us to process and forward logs. However, Fluent Bit has around 60 plugins and the functionality that you require might not be offered by any of them, or maybe you want to use a barebones version of a plugin. If this is the case, then Fluent Bit allows developers to write their own external plugin.

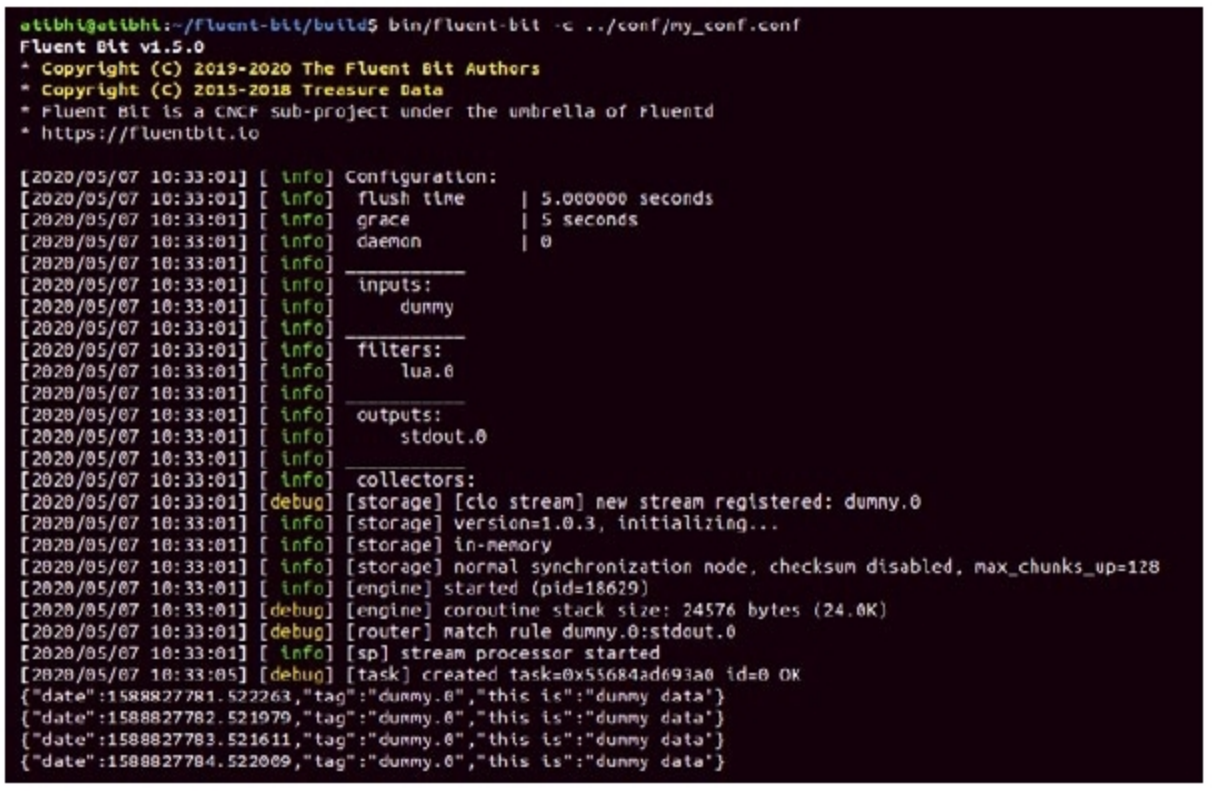

Figure 5: Output of the data pipeline setup

How to write your own external plugin

The Fluent Bit plugin repository *(https://github.com/fluent/fluent-bit-plugin)* contains examples of external plugins. First, we create a directory for our plugin's name and create a *CMakelists.txt* file in it. This contains the path to the plugin's main file and the plugin name:

```
set ( src my_plugin.c )
FLB_PLUGIN ( my_plugin "${src}" "" )
```

Then we create a main file for our plugin, i.e., *my_plugin.c* and this contains the code for our plugin. This file will have all the relevant headers and structure needed. A detailed guide on writing a plugin can be found at *https://github.com/fluent/fluent-bit/blob/38ad476486e90084da38668dadc789ca80ab17e1/DEVELOPER_GUIDE.md#plugin-api*.

After writing the main file of our plugin, we need to load it in Fluent Bit. We first add a plugins section in our *SERVICE* section and also add the name of our external plugin to the relevant section *(INPUT/OUTPUT/PARSER* or *FILTER)*.

```
[SERVICE]
# Path to the plugin configuration file
Plugins_File plugin.conf

[OUTPUT]
Name my_plugin
```

To build the plugin, we will first make sure we are inside the Fluent Bit build directory. Assume Fluent Bit is located at */tmp/fluent-bit* and run *CMake* with two important variables – *FLB_SOURCE* which is the absolute path to the Fluent Bit source code, and *PLUGIN_NAME* which is the directory name of the project that we want to build.

```
$ cmake -DFLB_SOURCE=/tmp/fluent-bit
-DPLUGIN_NAME=plugin_name ../
$ make
```

Now if we query the contents of the current directory, we can find a *.so* file. This is our dynamic plugin that can now be loaded from Fluent Bit.

```
$ ls -l *.so
-rwxr-xr-x 1 atibhi atibhi 17288 may 7
20:33 plugin.so
```

It can then be loaded using the *plugin.conf* file:

```
[PLUGINS]
# Path to the .so file of the external
plugin


Path /path/my_plugin.so
```

In this article we have explored what Fluent Bit is all about – how it processes logs, how to set up our data pipeline, and how to write our own external plugin. END

By: Atibhi Agrawal and Prof. B. Thangaraju

The authors are associated with the open source technology lab in the International Institute of Information Technology, Bengaluru. Atibhi Agrawal is an active contributor to the Fluent Bit project.